

Безопасность и управление доступом

Table of contents

Безопасность и управление доступом	1
Аутентификация vs Авторизация	1
RBAC (Role-Based Access Control)	2
ABAC (Attribute-Based Access Control)	3
JWT и OAuth 2.0	5
JSON Web Token (JWT)	5
OAuth 2.0 Authorization Code Flow	6
Single Sign-On (SSO)	8
SAML 2.0 (Security Assertion Markup Language)	8
OAuth 2.0 / OpenID Connect (OIDC)	8

Безопасность и управление доступом

С нагрузкой разобрались, теперь про **безопасность**. Без неё продакшен жить не может: любой запрос придётся проверять, кто его прислал и что ему позволено делать.

Базовые вопросы простые. Кто стучится в API? Точно ли он тот, за кого себя выдаёт? Что ему разрешено читать и менять? Если на эти вопросы нет ответа, посторонний пользователь спокойно удаляет чужие записи или подменяет личность.

Разберём три уровня защиты:

1. **Аутентификация и авторизация** (RBAC, ABAC) — проверка личности и прав.
2. **Токены и федеративная аутентификация** (JWT, OAuth 2.0) — как передавать права между системами.
3. **Single Sign-On (SSO)** — один логин на много сервисов (SAML, OpenID Connect).

В Google или Microsoft сотрудник не вводит пароль для каждого внутреннего сервиса: он логинится в корпоративный SSO и получает доступ ко всему сразу. Соцсети используют OAuth, чтобы сторонние приложения работали с данными пользователя без передачи пароля. Дальше разбираем, как такие схемы устроены изнутри.

Аутентификация vs Авторизация

Два слова легко путают, но отвечают они на разные вопросы.

Аутентификация (Authentication): кто вы?

- Проверка личности пользователя.
- Методы: пароль, OAuth, биометрия, 2FA.

Авторизация (Authorization): что вам разрешено делать?

- Проверка прав доступа.
- Методы: RBAC, ABAC, ACL.

Пример:

1. Аутентификация: пользователь `alice@company.com` вводит пароль → система проверяет → выдаёт токен
2. Авторизация: `alice` запрашивает `DELETE /api/users/123`
 → система проверяет: у `alice` есть роль `admin`?
 → если да → разрешить, иначе → `403 Forbidden`

RBAC (Role-Based Access Control)

Идея простая: права назначаются **ролям**, а пользователи получают роли. Никаких индивидуальных разрешений, всё через прослойку.

Пользователи → Роли → Права (Permissions)

Пример:

Alice → Role: Editor → Permissions: [read_posts, write_posts, delete_own_posts]
 Bob → Role: Admin → Permissions: [read_posts, write_posts, delete_any_posts, manage_users]
 Carol → Role: Viewer → Permissions: [read_posts]

Покажем на системе управления контентом. Роли и права:

```
type Permission string

const (
    ReadPosts      Permission = "posts:read"
    WritePosts      Permission = "posts:write"
    DeleteOwnPosts  Permission = "posts:delete:own"
    DeleteAnyPosts  Permission = "posts:delete:any"
    ManageUsers      Permission = "users:manage"
)

type Role struct {
    Name          string
    Permissions []Permission
}

var roles = map[string]Role{
    "viewer": {
        Name:          "Viewer",
        Permissions: []Permission{ReadPosts},
    },
}
```

```

    },
    "editor": {
        Name: "Editor",
        Permissions: []Permission{ReadPosts, WritePosts, DeleteOwnPosts},
    },
    "admin": {
        Name: "Admin",
        Permissions: []Permission{ReadPosts, WritePosts, DeleteOwnPosts,
DeleteAnyPosts, ManageUsers},
    },
}

```

Проверка доступа:

```

func (u *User) HasPermission(perm Permission) bool {
    role := roles[u.Role]
    for _, p := range role.Permissions {
        if p == perm {
            return true
        }
    }
    return false
}

// Пример использования
if !user.HasPermission(DeleteAnyPosts) {
    return errors.New("403 Forbidden")
}

```

Чем хорош RBAC:

- Управление простое: поменяли права у роли — изменилось у всех её носителей.
- Структуру прав легко удержать в голове.

Где начинает скрипеть:

- Динамические правила вида «редактировать только свои посты» так не выразить.
- На сложных требованиях ролей становится слишком много: под каждую комбинацию прав приходится заводить отдельную.

Где встречается чаще всего: веб-приложения, CMS, корпоративные системы.

ABAC (Attribute-Based Access Control)

Здесь решение о доступе строится на **атрибутах** субъекта, ресурса и контекста, а не на роли.

Субъект (User) + Ресурс (Resource) + Действие (Action) + Контекст (Context)
→ Политика (Policy) → Решение (Allow/Deny)

Атрибуты, которые приходят на вход политикам:

```
{
  "subject": {
    "id": "alice",
    "department": "engineering",
    "role": "senior",
    "clearance_level": 3
  },
  "resource": {
    "type": "document",
    "id": "doc-123",
    "classification": "confidential",
    "owner": "alice",
    "department": "engineering"
  },
  "action": "edit",
  "context": {
    "time": "2025-01-02T14:30:00Z",
    "ip": "192.168.1.10",
    "mfa_verified": true
  }
}
```

Примеры политик:

Политика 1: Пользователь может редактировать свои документы
IF subject.id == resource.owner
AND action == "edit"
THEN Allow

Политика 2: Старшие сотрудники могут читать конфиденциальные документы своего департамента
IF subject.role == "senior"
AND subject.department == resource.department
AND resource.classification == "confidential"
AND action == "read"
THEN Allow

Политика 3: Доступ к секретным документам только в рабочее время с MFA

```

IF resource.classification == "secret"
AND subject.clearance_level >= 3
AND context.time BETWEEN 09:00 AND 18:00
AND context.mfa_verified == true
THEN Allow
ELSE Deny

```

Чем хорош ABAC:

- Гибкость: выражаются сложные правила, которые в RBAC требуют десятка ролей.
- Динамические проверки по времени, IP, статусу MFA.
- Ролей плодится заметно меньше.

Цена:

- Реализация и отладка сложнее.
- На каждый запрос приходится вычислять политики, что бьёт по производительности.

Где встречается: банки и финансы со сложными правилами доступа, государственные системы с классификацией документов, healthcare (HIPAA, доступ по атрибутам пациента и врача).

Сравнение RBAC и ABAC:

Характеристика	RBAC	ABAC
Основа	Роли	Атрибуты (субъект, ресурс, контекст)
Сложность	Простая	Сложная
Гибкость	Низкая	Высокая
Динамические правила	Нет	Да (время, IP, MFA и т.д.)
Производительность	Быстро (проверка в памяти)	Медленнее (вычисление политик)
Когда использовать	Простые иерархии, CMS, админки	Сложные правила, финансы, healthcare

JWT и OAuth 2.0

JSON Web Token (JWT)

JWT — это токен с закодированными claims (утверждениями) о пользователе, подписанный сервером. Подпись гарантирует, что содержимое никто не подделал.

Структура:


```

    &redirect_uri=https://photoeditor.com/callback
    &scope=drive.readonly
    &response_type=code
    &state=RANDOM_CSRF_TOKEN          // защита от CSRF, вернётся в callback
    &code_challenge=BASE64URL_SHA256  // PKCE: хеш одноразового секрета
    &code_challenge_method=S256

```

3. Пользователь видит экран согласия Google:
 "PhotoEditor хочет получить доступ к вашим файлам на Drive"
 → Пользователь нажимает "Разрешить"

4. Google перенаправляет обратно на PhotoEditor с authorization code:
 GET https://photoeditor.com/callback?code=AUTH_CODE_HERE&state=RANDOM_CSRF_TOKEN
 // PhotoEditor сверяет, что state совпадает с тем, что он отправил на шаге 2.

5. PhotoEditor (серверная часть) обменивает code на access token:
 POST https://oauth2.googleapis.com/token
 {
 "code": "AUTH_CODE_HERE",
 "client_id": "...",
 "client_secret": "...", // секрет, известный только серверной части
 "redirect_uri": "...",
 "grant_type": "authorization_code",
 "code_verifier": "..." // PKCE: оригинал секрета, проверяется против challenge
 }

6. Google возвращает access token:
 {
 "access_token": "ya29.a0AfH6SMC...",
 "refresh_token": "1//0gX...",
 "expires_in": 3600,
 "token_type": "Bearer"
 }

7. PhotoEditor использует access token для запросов к Google Drive API:
 GET https://www.googleapis.com/drive/v3/files
 Authorization: Bearer ya29.a0AfH6SMC...

Роли в OAuth 2.0:

- **Resource Owner** — пользователь (владелец фотографий).
- **Client** — приложение PhotoEditor.

- **Authorization Server** — Google, выдаёт токены.
- **Resource Server** — Google Drive API, хранит данные.

Single Sign-On (SSO)

Логинимся один раз — получаем доступ ко всем приложениям компании. Пароль не вводится для каждого сервиса отдельно.

```
Сотрудник входит в корпоративный портал (portal.company.com)
→ Автоматически получает доступ к:
- Email (mail.company.com)
- CRM (crm.company.com)
- Jira (jira.company.com)
- Confluence (wiki.company.com)
```

SAML 2.0 (Security Assertion Markup Language)

Enterprise-стандарт, работает через XML-сообщения. Старый, тяжёлый, но прочно вросший в корпоративный мир.

Flow:

```
1. Пользователь → открывает CRM (Service Provider)
2. CRM видит, что пользователь не залогинен → перенаправляет на SSO (Identity Provider)
3. SSO → показывает форму логина
4. Пользователь вводит credentials → SSO проверяет
5. SSO → создаёт SAML Assertion (XML с данными пользователя) и подписывает
6. SSO → перенаправляет обратно на CRM с SAML Assertion
7. CRM → проверяет подпись, извлекает данные пользователя → создаёт сессию
```

Где встречается: корпоративные системы, B2B-интеграции (Okta, Azure AD, Google Workspace).

OAuth 2.0 / OpenID Connect (OIDC)

Современная альтернатива на JSON Web Tokens. Формула короткая:

OpenID Connect = OAuth 2.0 + id_token (JWT с данными пользователя)

Flow:

```
1. Пользователь → открывает App
2. App → перенаправляет на Auth0 (OIDC Provider):
   GET /authorize?client_id=...&scope=openid profile email&response_type=code
3. Auth0 → показывает логин (или использует существующую сессию SSO)
4. Auth0 → перенаправляет обратно с authorization code
```


5. App → обменивает code на токены:

Получает: { access_token, id_token (JWT), refresh_token }

6. App → валидирует id_token и только потом доверяет claims:

- подпись по публичному ключу провайдера (JWKS);
- iss — действительно ли это наш OIDC-провайдер;
- aud — токен выписан именно нашему client_id;
- exp — не истёк;
- nonce — совпадает с тем, что мы отправили на /authorize.

После валидации App читает claims:

```
{
  "sub": "user-123",
  "email": "alice@company.com",
  "name": "Alice",
  "picture": "https://...",
  "iss": "https://auth0.company.com"
}
```

Декодировать JWT мало — без проверки подписи и iss/aud/exp/nonce любой может подсунуть приложению свой токен. Эти проверки делает SDK провайдера (Auth0, Okta), но знать, что они там, обязательно.

Где встречается: современные SaaS, мобильные приложения (Auth0, Okta, Keycloak).

Сравнение SAML и OAuth/OIDC:

Характеристика	SAML 2.0	OAuth 2.0 / OIDC
Формат	XML	JSON (JWT)
Сложность	Высокая	Средняя
Мобильные приложения	Плохо поддерживается	Отлично
API	RESTful wrapper нужен	Нативно RESTful
Enterprise adoption	Высокая (legacy системы)	Растёт (новые системы)
Когда использовать	B2B интеграции, enterprise SSO	SaaS, мобильные приложения, API